

Перечисления или "Enums" в PHP. Конспект.

- тип перечисления внутренне представлен как класс
- могут содержать методы и константы (свойства не могут)
- могут реализовывать интерфейсы и трейты
- наследование не допускается
- запрещены магические методы, кроме call, callStatic, и invoke
- клонирование не поддерживается

Чистые перечисления

```
enum Colors
{
    case Red;
    case Blue;
    case Green;

    public const COLOR_RED = self::Red;

    public function getColor(): string
    {
        // name - имя варианта (доступно только для чтения)
        return $this->name;
    }
}

// варианты реализованы как объекты
echo Colors::Red->getColor(); // "Red"
```

Типизированные перечисления

- может поддерживаться только одним типом int или string
- все варианты должны уникальнй скалярный эквивалент
- объявление интерфейса идёт после объявления типа

```
enum Colors: string implements Colorful
{
    case Red = 'красный';
    case Blue = 'синий';
    case Green = 'зелёный';
}

// value - скалярный эквивалент (доступно только для чтения)
print Colors::Red->value; // "красный"
```

Встроенные методы работы над перечислениями

```
// вернёт соотв-ий вариант. Если вариант не найден, метод выдаст ValueError
var_dump(Colors::from('красный')); // enum(Colors::Red)

// вернёт соотв-ий вариант. Если вариант не найден, метод вернёт null
var_dump(Colors::tryFrom('красный')); // enum(Colors::Red)

// список значений
var_dump(Colors::cases()); // array(3) { [0]=> enum(Colors::Red) [1]=> enum(Colors::Blue) [2]=> enum(C
```

Сериализация ENUMS

- сериализуются иначе, чем объекты
- есть новый код сериализации, "E", который указывает имя варианта перечисления
- если чистое перечисление сериализуется в JSON, будет выдана ошибка
- типизированное перечисление в JSON будет представлено скаляром значений
- поведение способов сериал-ии может быть изменено путём реализации JsonSerializable

Интерфейсы

- не могут называться одинаково с классами и трейтами
- могут содержать только пустые public-методы и константы (после PHP 8.1 константы м.б. переопределены в классе)
- могут определять магические методы
- могут наследоваться от других интерфейсов (причем нескольких)
- класс может реализовать несколько инт-в

```
interface Template extends A, B
{
    public function setVariable($name, $var);
    public function getHtml($template);
}

class Two extends One implements Template, Template2
{
    // методы от интерфейсов обязательны для реализации
    // ...
}
```

Магические методы - переопределяют действия по умолчанию над объектами

Перегрузка - создание динамически

```
// Вы-ся при записи в недоступные свойства
__set()

// Выз-ся при чтении недоступного св-ва
__get()

// Выз-ся при использовании isset() или empty() на недоступных св-вах
__isset()

// Выз-ся при использовании unset() на недоступном свойстве
__unset()
```

Перегрузка методов

```
// Запускается при нестатическом вызове недоступных методов
__call()

// Запускается при статическом вызове недоступных методов
static __callStatic()
```

Остальные магические методы

```
// Выз-ся при создании объекта
// Для исполь-я родительского констр-ра требуется вызвать parent::__construct()
// освобождается от правил совместимости сигнатуры при наследовании методов
__construct()

// Выз-ся при освобождении всех ссылок на объект или при завершении скрипта
// Порядок выполнения деструкторов не гарантируется
// Для вызова деструктора родительского класса, требуется вызвать parent::__destruct()
__destruct()

// Выз-ся перед сериализацией объекта
// Испол-зя для удобного представления сериализованного объекта
__serialize()

// Выз-ся перед сериализацией объекта, если нет метода __serialize()
// Испол-ся для завершения работы над данными, ждущими обработки
__sleep()

// Выз-ся перед десериализацией
__unserialize()

// Выз-ся перед десериализацией, если нет метода __unserialize()
// Испол-ся для восстановления соединений с базой данных
__wakeup()

// Выз-ся при попытке преобразовать объект к строке
// При испол-ии рекоменд-ся явно реализовать интерфейс Stringable
__toString()

// Выз-ся при попытке выполнить объект как функцию
__invoke()

// Выз-ся при использовании ключев. слова "clone"
__clone()

static __set_state()
// Выз-ся для классов, экспортированных функцией var_export()
// var_export – Выводит или возвращает строковое представление данных

__debugInfo()
// Выз-ся функцией var_dump()
// Если метод не определён, тогда будут выведены все свойства объекта
```

Генераторы в PHP. Конспект

[ссылка на документацию](#)

- Позволяют сэкономить память (по сравнению, например, с сбором данных в массиве) и структурировать код.

- Отличаются от обычных функций оператором `yield`.
- `yield` позволяет выйти из функции, но сохранить её состояние. При следующем вызове возвращается следующее значение.
- `yield` возвращает итерируемый объект `Generator` (implements `Iterator`).
- Генераторы нельзя перемотать назад после старта итерации (в отличие от итераторов).

```
$generator = (function() {  
    yield 1;  
    yield 2;  
  
    return 3;  
})();  
  
foreach ($generator as $value) {  
    echo $value, PHP_EOL; // 1, 2, 3  
}  
  
// или  
echo $generator->getReturn(), PHP_EOL; // 1, 2, 3
```

Методы работы с генераторами

```
$generator->current(); // Получить текущее значение  
$generator->key();     // Получить все значения по порядку  
$generator->next();     // Возобновить работу генератора  
$generator->send($value); // Передать значение в генератор  
$generator->rewind();    // Перемотать итератор  
$generator->throw($exc); // Бросить исключение в генератор  
$generator->valid();     // Проверка, закрыт ли итератор  
$generator->__wakeup();  // Callback-функция сериализации
```